

PORTADA

Arquitectura de Software

Semana 4 — El Rol del Arquitecto de Software

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

3 días
sin poder enviar un solo DVD



CASO · FICHA TÉCNICA

Netflix, agosto de 2008

Empresa	Netflix — en 2008, todavía principalmente un servicio de alquiler de DVD por correo
Incidente	Corrupción de la base de datos relacional central
Impacto	Netflix no pudo enviar DVDs a sus clientes durante 3 días
Causa raíz	Arquitectura con un único punto de falla : toda la operación dependía de una sola base de datos centralizada
Rol creado/asignado	Cloud Systems Architect — liderado por Adrian Cockcroft desde 2007–2008
Resultado final	Migración completa de los sistemas de streaming a AWS, con arquitectura de microservicios, anunciada terminada en 2016

Caso ampliamente documentado en charlas públicas y publicaciones de ingeniería de Netflix (Adrian Cockcroft, "Netflix Tech Blog").

CASO · LO QUE FALLÓ

Un solo punto, toda la operación

La base de datos relacional que se corrompió en 2008 no era "una parte más" del sistema: **era el único lugar donde vivían los datos necesarios para procesar y enviar cada pedido.**

Ningún componente podía seguir funcionando sin ella. No había redundancia, ni forma de aislar el fallo, ni un camino alternativo.

El problema no era "un bug" en ese componente — era que la arquitectura entera dependía de un único punto que podía fallar.

CASO · EL ROL QUE SE NECESITÓ

No un programador más — alguien que viera el sistema completo

Resolver esto no era responsabilidad de un solo equipo arreglando su propio módulo. Se necesitaba a alguien capaz de:

- **Diagnosticar la causa estructural** — no el síntoma puntual, sino la falta de redundancia en toda la arquitectura.
- **Definir principios de diseño para todos los equipos:** desacoplar servicios, eliminar puntos únicos de falla, diseñar asumiendo que los componentes van a fallar.
- **Coordinar la migración** entre decenas de equipos, durante años, sin detener el negocio.

Ese conjunto de responsabilidades es, en esencia, el trabajo de un arquitecto de software — en este caso, con el título específico de Cloud Architect.

CASO · EL RESULTADO

De un punto único de falla a resiliencia distribuida

La migración tomó años, no semanas. Para 2016, Netflix había trasladado por completo sus sistemas de streaming a AWS, organizados como **cientos de microservicios independientes**, cada uno capaz de fallar sin tumbar el sistema completo.

Netflix incluso desarrolló herramientas para provocar fallos deliberadamente en producción (el famoso "Chaos Monkey") — una consecuencia directa de la nueva filosofía arquitectónica: si el sistema no puede sobrevivir a un fallo aislado, la arquitectura sigue estando mal.

TRANSICIÓN

¿Quién toma decisiones de este tamaño?

Cockcroft no escribió personalmente cada línea de los cientos de microservicios de Netflix. Su trabajo fue distinto: **ver el sistema completo, definir principios, y guiar a decenas de equipos hacia una misma dirección estructural.**

Ese es exactamente el rol que estudiamos hoy: qué hace un arquitecto de software, qué tipos existen, y qué cualidades lo distinguen de otros roles técnicos.

CONCEPTO

¿Qué es un arquitecto de software?

Un **arquitecto de software** es la persona (o el rol) responsable de tomar y comunicar las **decisiones estructurales** de un sistema — las que vimos en la Semana 3 que son costosas de revertir y afectan al sistema completo.

No es necesariamente un título jerárquico ni el puesto de mayor antigüedad. Es, ante todo, un rol técnico: quien asume la responsabilidad de que la estructura del sistema sea coherente, sostenible y comunicable a todo el equipo.

CONCEPTO

Responsabilidades típicas

- **Tomar decisiones técnicas de alto nivel:** patrones arquitectónicos, tecnologías centrales, estructura de componentes.
- **Comunicar esas decisiones** a stakeholders técnicos y no técnicos — desde desarrolladores hasta dirección de negocio.
- **Balancear restricciones en tensión:** calidad, costo, tiempo de entrega, y los requisitos no funcionales que se estudian el jueves.
- **Guiar y mentorear** a los equipos de desarrollo en la aplicación consistente de esas decisiones.
- **Revisar y ajustar** la arquitectura a medida que el sistema evoluciona, sin perder de vista el conjunto.

CONCEPTO

Tipos de arquitecto

Tipo	Alcance de sus decisiones
Arquitecto de Software / Aplicación	La estructura interna de una aplicación específica
Arquitecto de Soluciones	Cómo varios sistemas se integran para resolver una necesidad de negocio
Arquitecto Empresarial	Toda la cartera de sistemas de una organización, alineada a su estrategia
Arquitecto de Datos	Cómo se modelan, almacenan y fluyen los datos entre sistemas
Arquitecto de Nube (Cloud Architect)	Cómo se despliega y escala el sistema en infraestructura en la nube — el rol de Cockcroft en Netflix

CONTRAEJEMPLO

Dos mitos comunes sobre el rol

Mito 1: "el arquitecto es quien más código escribe." El valor del rol está en las decisiones estructurales y en cómo se comunican — no en el volumen de código producido personalmente.

Mito 2: "arquitecto es un título jerárquico, viene con antigüedad." Es un rol definido por el tipo de decisión que se toma, no por el tiempo en la empresa — alguien con pocos años de experiencia puede asumir decisiones arquitectónicas en un proyecto pequeño, como el Sprint 1 de este curso.

CONCEPTO

Cualidades técnicas

- **Visión de sistemas:** capacidad de ver cómo interactúan las partes, no solo cómo funciona cada una por separado.
- **Conocimiento amplio de tecnologías:** no experto en todas, pero con criterio para evaluar cuándo cada una es apropiada.
- **Experiencia con las consecuencias de decisiones pasadas:** haber visto qué falla y por qué, en sistemas reales.
- **Rigor con los requisitos no funcionales:** escalabilidad, seguridad, mantenibilidad — se profundiza el jueves.



CONCEPTO

Cualidades no técnicas

- **Comunicación clara:** traducir decisiones técnicas complejas a distintas audiencias.
- **Liderazgo técnico sin autoridad formal:** influir en equipos que no reportan jerárquicamente al arquitecto.
- **Capacidad de decidir bajo incertidumbre:** como se vio en la Semana 3, las decisiones arquitectónicas se toman temprano, con información incompleta.
- **Mentoría:** elevar el criterio técnico del equipo, no solo imponer decisiones.



CONCEPTO · APLICADO AL CASO

Lo que hizo Cockcroft, con nombre propio

Visión de sistemas	Identificó que el problema no era un componente, sino la falta de redundancia en toda la arquitectura
Decisión bajo incertidumbre	Comprometió a Netflix con una migración de años, sin garantía previa de que funcionaría a esa escala
Comunicación y liderazgo	Alineó a decenas de equipos autónomos hacia los mismos principios arquitectónicos



TENSIÓN CONCEPTUAL

¿El arquitecto debe seguir programando?

Postura 1: un arquitecto que deja de programar pierde contacto con la realidad del código y termina tomando decisiones poco prácticas.

Postura 2: las responsabilidades de comunicación, coordinación y visión de sistema —como las de Cockcroft durante la migración de Netflix— exigen dedicación de tiempo completo, incompatible con programar a diario.

¿Puede alguien ser un buen arquitecto sin escribir código regularmente?



INTERACCIÓN

¿Qué tipo de arquitecto se necesita?

Para cada escenario, identifiquen qué tipo de arquitecto (de la tabla de tipos) sería el más apropiado para liderar la decisión:

1. Definir cómo se van a comunicar 5 aplicaciones distintas de una universidad entre sí.
2. Decidir cómo se escala automáticamente el backend de un sistema durante un evento de alta demanda.
3. Diseñar cómo se van a almacenar y sincronizar los datos de estudiantes entre tres sistemas distintos.
4. Definir la estructura interna de módulos del proyecto de Fábrica Escuela.

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. Un **arquitecto de software** es responsable de las decisiones estructurales del sistema, no del título jerárquico ni del volumen de código escrito.
2. Existen distintos **tipos de arquitecto** (software/aplicación, soluciones, empresarial, datos, nube) según el alcance de sus decisiones.
3. Las cualidades del rol combinan lo **técnico** (visión de sistemas, criterio tecnológico) con lo **no técnico** (comunicación, liderazgo sin autoridad formal).
4. El caso Netflix (2008–2016) muestra el rol en acción: diagnóstico estructural, decisión bajo incertidumbre, y coordinación a largo plazo.
5. No hay consenso único sobre si un arquitecto debe seguir programando activamente — depende del contexto y la escala del sistema.

INSTRUCCIONES

Taller de hoy — Implementación de ejemplo

Demostración guiada por el docente — 30 a 40 minutos

El docente muestra en vivo, sobre el mismo backend Spring Boot de sesiones anteriores, **cómo eliminar un punto único de falla** — el mismo tipo de decisión que enfrentó Netflix en 2008, a escala reducida.

De un único punto de falla a redundancia

```
// ANTES: un solo repositorio, sin manejo de fallos.  
// Si la fuente de datos falla, todo el sistema falla.  
public class SaludoService {  
    @Autowired SaludoRepository repo;  
    public String generar() {  
        return repo.buscarMensaje(); // sin plan B  
    }  
}  
  
// DESPUÉS: decisión arquitectónica de resiliencia,  
// con una fuente de respaldo ante fallo del repositorio.  
public class SaludoService {  
    @Autowired SaludoRepository repoPrincipal;  
    @Autowired SaludoRepositoryRespaldo repoRespaldo;  
  
    public String generar() {  
        try {  
            return repoPrincipal.buscarMensaje();  
        } catch (Exception fallo) {  
            return repoRespaldo.buscarMensaje(); // continúa operando  
        }  
    }  
}
```

No hay entregable de estudiantes: el objetivo es reconocer, a pequeña escala, la misma decisión estructural que tomó Netflix en 2008.

CIERRE

Para la próxima sesión

- Pensar qué tipo de arquitecto sería más relevante para el proyecto propio de Fábrica Escuela, y por qué.
- Repasar la distinción entre requisito funcional y no funcional, si ya la conocen de cursos anteriores — se formaliza el jueves.

El jueves estudiamos los Requisitos No Funcionales bajo la norma ISO 25010 — la herramienta que todo arquitecto necesita para tomar decisiones defendibles, no intuitivas.

